

TOY ISA 5

Instruction Set Architecture Specification

Proteus Lab

Generated on: 2026-09-17

Version: 1.5

Table of Contents

Architecture Overview	3
ST	4
ADDI	5
OR	6
LD (register)	7
J	8
BEQ	9
CLZ	10
SSAT	11
LD	12
ADD	13
SYSCALL	14
BEXT	15
LI	16
RORI	17
STP	18

Architecture Overview

Register file and ABI

X0 return value	X1 arg 0	X2 arg 1	X3 arg 2
X4 arg 3	X5 arg 4	X6 arg 5	X7 arg 6
X8 syscall number	X9	X10	X11
X12	X13	X14	X15
X16	X17	X18	X19
X20	X21	X22	X23
X24	X25	X26	X27
X28	X29	X30	X31 LR

32 registers, 32-bit each

PC program counter

Memory

Memory is byte-addressed and little-endian. A word is 32 bits wide.

ST

Encoding

31:26	25:21	20:16	15:14	13:0
100000	base	rt	00	imm

Assembler

```
ST rt, [base, #imm]
```

Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt]
```

ADDI

Encoding

31:26	25:21	20:16	15:0
010111	rs	rt	imm

Assembler

```
ADDI rt, rs, #imm
```

Semantics

```
 $X[rt] \leftarrow X[rs] + \text{sign\_extend}(imm)$ 
```

OR

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rs	rt	rd	00000	011010

Assembler

```
OR rd, rs, rt
```

Semantics

```
 $X[rd] \leftarrow X[rs] \mid X[rt]$ 
```

LD (register)

Encoding

31:26	25:21	20:16	15:14	4:0	13:5
000011	base	rt	11	rm	000000000

Assembler

```
LD rt, [base, rm]
```

Semantics

```
addr ← X[base] + X[rm]
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
X[rt] ← memory[addr]
```

J

Encoding

31:26	25:0
111111	instr_index

Assembler

J target

Semantics

$PC \leftarrow (PC \ \& \ 0xF0000000) \mid (instr_index \ll 2)$

BEQ

Encoding

31:26	25:21	20:16	15:0
010000	rs	rt	offset

Assembler

```
BEQ rs, rt, #offset
```

Semantics

```
target ← sign_extend(offset) << 2
```

```
cond ← X[rs] == X[rt]
```

```
PC ← if (cond) PC + target else PC + 4
```

CLZ

Encoding

31:26	25:21	20:16	15:6	5:0
000000	rd	rs	0000000000	110010

Assembler

```
CLZ rd, rs
```

Semantics

```
 $X[rd] \leftarrow \text{count\_leading\_zeros}(X[rs])$ 
```

Notes

Counts the number of leading zero bits in the $X[rs]$ register.

SSAT

Encoding

31:26	25:21	20:16	15:11	10:0
001000	rd	rs	imm5	000000000000

Assembler

```
SSAT rd, rs, #imm5
```

Semantics

```
 $X[rd] \leftarrow \text{saturate\_signed}(X[rs], \text{imm5})$ 
```

Notes

Saturation of the signed value in $x[rs]$ to N bits, where $N = \#imm$

LD

Encoding

31:26	25:21	20:16	15:14	13:0
110011	base	rt	00	imm

Assembler

```
LD rt, [base, #imm]
```

Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
X[rt] ← memory[addr]
```

ADD

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rs	rt	rd	00000	001100

Assembler

```
ADD rd, rs, rt
```

Semantics

```
 $X[rd] \leftarrow X[rs] + X[rt]$ 
```

SYSCALL

Encoding

31:26	25:6	5:0
000000	code	011110

Assembler

SYSCALL

Semantics

SigException(SystemCall)

Notes

x8 — system call number, x0 - x7 — args, x0 — result, see man syscall

BEXT

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rd	rs1	rs2	00000	110100

Assembler

```
BEXT rd, rs1, rs2
```

Semantics

```
 $X[rd] \leftarrow \text{bit\_extract}(X[rs1], X[rs2])$ 
```

Notes

Extracts and packs bits from register $X[rs1]$ into the positions where the corresponding bits in mask $X[rs2]$ are set. The extracted bits will place the least significant bits of the result Rd .

LI

Encoding

31:26	25:21	20:16	15:0
001011	00000	rt	imm

Assembler

```
LI rt, #imm
```

Semantics

```
 $X[rt] \leftarrow \text{sign\_extend}(\text{imm})$ 
```

RORI

Encoding

31:26	25:21	20:16	15:11	10:0
011101	rd	rs	imm5	0000000000

Assembler

```
RORI rd, rs, #imm5
```

Semantics

```
 $X[rd] \leftarrow \text{rotate\_right}(X[rs], \text{imm5})$ 
```

Notes

Rotates the bits of the $X[rs]$ register to the right by $\#imm$ bits

STP

Encoding

31:26	25:21	20:16	15:11	10:0
010101	base	rt1	rt2	offset

Assembler

```
STP rt1, rt2, offset(base)
```

Semantics

```
addr ← X[base] + sign_extend(offset)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt1]
```

```
memory[addr + 4] ← X[rt2]
```